

1. Object क्या होता है ?

- JavaScript में Object एक data structure होता है, जिसमें key-value pair के रूप में data store किया जाता है।
- Object का उपयोग real-world entity को represent करने के लिए किया जाता है।
- उदाहरण: Student, Car, Book आदि।

Object का सिंटैक्स:

```
let objectName = {  
  key1 : value1,  
  key2 : value2,  
  key3 : value3  
};
```

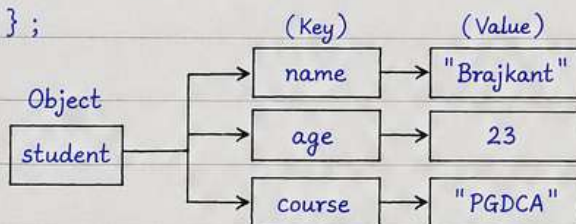
← Start of Object

← Key-Value Pair

← End of Object

उदाहरण:

```
let student = {  
  name : "Braj Kant",  
  age : 23,  
  course : "PGDCA"  
};
```



* Note:

Object unordered होता है, लेकिन ES6 के बाद insertion order maintain किया जाता है।

Example (1)

Object Literal

```
let student = {  
  name : "Braj Kant",  
  age : 23,  
  course : "PGDCA"  
};  
console.log(student);  
// Output: { name: 'Braj Kant', age: 23, course: 'PGDCA' }
```

Key Points (1)

- Object key-value pair में data store करता है।
- Key unique होती है।
- Value किसी भी type की हो सकती है।
- Object का उपयोग real-world data को model करने के लिए किया जाता है।

2. Object की विशेषताएँ

- Object unordered होता है।
- Duplicate keys नहीं हो सकती।
- Key string या Symbol type की होती है।
- Value किसी भी type की हो सकती है (string, number, array, function, object आदि)
- Object mutable होता है (इसके values बदले सकते हैं)।
- एक Object के अंदर function (method) को store किया जा सकता है।

उदाहरण:

```
let person = {  
  name : "Braj Kant",  
  age : 23,  
  isStudent : true,  
  skills : ["HTML", "CSS", "JavaScript"],  
  address : { city : "Kanpur", state : "UP" },  
  sayHello : function() {  
    console.log("Hello, I am Braj Kant");  
  }  
};
```

* इसमें name, age, isStudent, skills, address, sayHello ये सभी keys हैं और इनके सामने जो values हैं, वे इनके values हैं।

Example (2)

new Object() Constructor

```
let student = new Object();  
student.name = "Braj Kant";  
student.age = 23;  
student.course = "PGDCA";  
console.log(student);  
// Output: { name: 'Braj Kant', age: 23, course: 'PGDCA' }
```

Key Points (2)

- Object unordered होता है लेकिन insertion order maintain करता है।
- Object mutable होता है।
- Object में function को store किया जा सकता है।
- Object complex data को store करने में सहायक है।



3. Object बनाने के तरीके

JavaScript में Object बनाने के कई तरीके होते हैं।

(i) Object Literal

यह सबसे आसान और ज्यादा उपयोग किया जाने वाला तरीका है।

Syntax:

```
let obj = { key1: value1, key2: value2, ... };
```

Example:

```
let student = {  
  name: "Braj Kant",  
  age: 23,  
  course: "PGDCA"  
};
```

(iii) new Object() Constructor

new Object() Constructor का उपयोग करके भी Object बनाया जा सकता है।

Syntax:

```
let obj = new Object();
```

Example:

```
let student = new Object();  
student.name = "Braj Kant";  
student.age = 23;  
student.course = "PGDCA";
```

(iii) Object.create() Method

Object.create() method एक existing object को prototype के रूप में लेकर नया object बनाता है।

Syntax:

```
let obj = Object.create(proto);
```

Example:

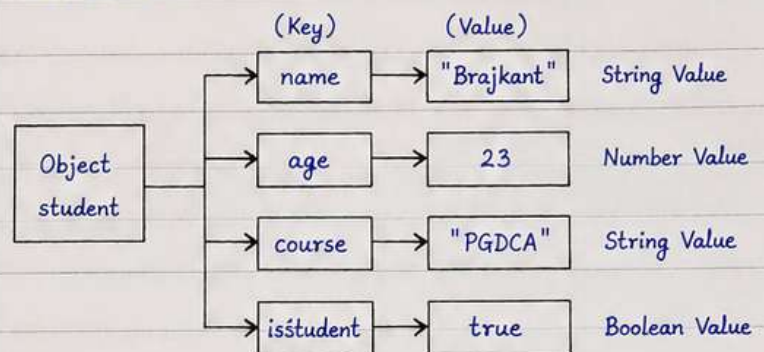
```
let person = {  
  type: "Human"  
};  
let student = Object.create(person);  
student.name = "Braj Kant";  
student.age = 23;  
student.course = "PGDCA";
```

* Note:

- Object एक Non-Primitive data type है।
- Object में data key-value pair के रूप में store होता है।
- Object mutable होते हैं, यानी हम इनके values को बदल सकते हैं।

4. Object के तत्व (Components)

JavaScript Object मुख्य रूप से चार चीजों से मिलकर बना होता है:



(i) Key (Property Name): Object के तत्व का नाम होता है। यह String या Symbol type का होता है।

(ii) Value (Property Value): किसी key के साथ जुड़ा हुआ data value होता है। यह किसी भी type का हो सकता है।

(iii) Key-Value Pair: हर key के साथ उसकी value होती है, जिसे key-value pair कहते हैं।

(iv) Object Name (Reference): Object को access करने के लिए एक नाम (reference) होता है।

Example:

```
let student = {  
  name: "Braj Kant", // key: name, value: "Braj Kant"  
  age: 23,           // key: age, value: 23  
  course: "PGDCA",  // key: course, value: "PGDCA"  
  isStudent: true   // key: isStudent, value: true  
};
```

Key Points

- Object key unique होता है।
- Key String या Symbol type का होता है।
- Value किसी भी data type की हो सकती है (String, Number, Boolean, Array, Function, Object आदि)।
- Object unordered होता है (insertion order maintain होता है ES6 के बाद)।
- Object mutable होता है (values बदल सकते हैं)।



5. Object ke ख़तमे Operations

Object ke saath hum kai operations kar sakte hain jaise property add karna, access karna, update karna aur delete karna.

(i) Add Property

Object me nayi property add karne ke liye dot notation ya bracket notation ka use karte hain.

Example:

```
let student = {  
  name: "Brajkant"  
};  
student.age = 23; // new property add  
student["course"] = "PGDCA";  
console.log(student);
```

(ii) Access Property

Object ki property ko access karne ke liye dot notation ya bracket notation ka use hota hai.

Example:

```
console.log(student.name); // Brajkant  
console.log(student.age); // 23  
console.log(student["course"]); // PGDCA
```

(iii) Update Property

Agar property pehle se exist karti hai to uske value ko update kiya ja sakta hai.

Example:

```
student.age = 24; // update value  
student.course = "Web Development";
```

(iv) Delete Property

Object ki kisi property ko delete karne ke liye delete keyword ka use hota hai.

Example:

```
delete student.age;  
delete student["course"];  
console.log(student);
```

Key Points

- Object me properties dynamically add, update aur delete ki ja sakti hain.
- Dot notation zyada use hota hai.
- Bracket notation tab use hota hai jab key string me ho ya variable ke through ho.
- delete keyword property ko object se hata deta hai.

6. Dot Notation vs Bracket Notation

Object ki properties ko access karne ke liye do methods hote hain - Dot Notation aur Bracket Notation.

(i) Dot Notation

- Isme property ka naam directly dot (.) ke baad likha jata hai.
- Jab key ek valid identifier ho (jaise name, age) tab dot notation use karte hain.
- Ye likhne me simple aur readable hota hai.

Example:

```
let student = { name: "Brajkant", age: 23 };  
console.log(student.name); // Brajkant  
console.log(student.age); // 23
```

(ii) Bracket Notation

- Isme property ka naam square bracket [] ke andar string ke form me likha jata hai.
- Jab key me space ho, special character ho, ya variable ke through access karna ho tab bracket notation use karte hain.
- Ye zyada flexible hota hai.

Example:

```
let student = {  
  name: "Brajkant",  
  "full name": "Brajkant Sharma",  
  age: 23  
};  
console.log(student["name"]); // Brajkant  
console.log(student["full name"]); // Brajkant Sharma  
let key = "age";  
console.log(student[key]); // 23
```

Dot Notation vs Bracket Notation

Basis	Dot Notation	Bracket Notation
Syntax	object.key	object["key"]
Key type	Only valid identifier	String, variable, any key
Use case	Simple keys	Keys with space, special character or dynamic keys
Readability	More readable	Less readable
Example	student.name	student["full name"]

7. Object Methods

Object के अंदर function को method कहते हैं।
Method object के data पर काम करता है।

Syntax:

```
let objectName = {  
  key1 : value1,  
  key2 : value2,  
  methodName : function () {  
    // code  
  }  
};
```

Example:

```
let student = {  
  name : "Brajkant",  
  age : 23,  
  course : "PGDCA",  
  displayInfo : function () {  
    console.log("Name : " + this.name);  
    console.log("Age : " + this.age);  
    console.log("Course : " + this.course);  
  }  
};
```

// Method Call

student.displayInfo();

Output:

```
Name : Brajkant  
Age : 23  
Course : PGDCA
```

Example 2:

```
let calculator = {  
  num1 : 10,  
  num2 : 5,  
  add : function () {  
    return this.num1 + this.num2;  
  },  
  multiply : function () {  
    return this.num1 * this.num2;  
  }  
};  
console.log(calculator.add()); // 15  
console.log(calculator.multiply()); // 50
```

Key Points

- Method object के अंदर define होता है।
- Method object के data (properties) पर काम करता है।
- Method call करने पर execute होता है।
- this keyword method के अंदर current object को refer करता है।

8. this Keyword

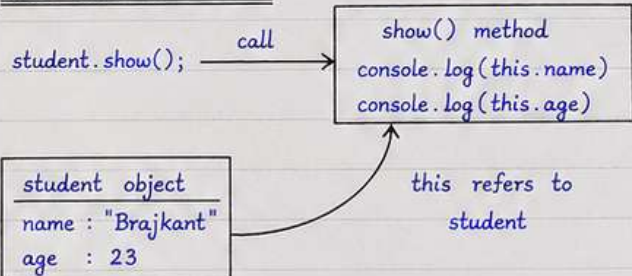
this keyword method के अंदर current object को refer करता है।

Yah batata hai ki method kis object के liye call हो रहा है।

Example 1:

```
let student = {  
  name : "Brajkant",  
  age : 23,  
  show : function () {  
    console.log("Name : " + this.name);  
    console.log("Age : " + this.age);  
  }  
};  
student.show(); // Output : Name : Brajkant  
                // Age : 23
```

Kaise kaam karta hai?



Example 2:

```
let user = {  
  name : "Brajkant",  
  greet : function () {  
    console.log("Hello " + this.name + "! Welcome");  
  }  
};  
user.greet(); // Hello Brajkant! Welcome
```

Example 3:

```
function printThis() {  
  console.log(this);  
}  
printThis(); // Global object (window)  
let obj = {  
  name : "BK",  
  showThis : printThis  
};  
obj.showThis(); // obj object
```

Key Points

- this method के अंदर current object को refer करता है।
- this का use properties aur methods को access करने के liye होता है।
- this हर jagah alag object को refer कर सकता है, जहाँ से function call किया जाता है।



9. Object Destructuring (वस्तु विखंडन)

Object Destructuring ES6 का एक feature है, जिसके द्वारा Object की properties को अलग-अलग variables में आसानी से निकाला जा सकता है।

Syntax :

```
let { property1, property2, ... } = objectName;
```

Example 1 : Basic Destructuring

```
let student = {
  name : "Brajkant",
  age : 23,
  course : "PGDCA",
  city : "Kanpur"
};
// Destructuring
let { name, age, course, city } = student;
console.log(name); // Brajkant
console.log(age); // 23
console.log(course); // PGDCA
console.log(city); // Kanpur
```

Example 2 : Destructuring with Different Names

```
let { name : studentName, age : studentAge } = student;
console.log(studentName); // Brajkant
console.log(studentAge); // 23
```

Example 3 : Default Values

```
let { name, age, branch = "IT" } = student;
console.log(branch); // IT (default)
```

Example 4 : Nested Object Destructuring

```
let studentDetail = {
  name : "Brajkant",
  address : {
    city : "Kanpur",
    state : "Uttar Pradesh"
  }
};
// Nested Destructuring
let { name, address : { city, state } } = studentDetail;

console.log(name); // Brajkant
console.log(city); // Kanpur
console.log(state); // Uttar Pradesh
```

Key Points

- Destructuring से code छोटा और clean बनता है।
- Properties को अलग-अलग variables में extract किया जाता है।
- Nested object में भी destructuring संभव है।
- Default value देते समय, यदि property मौजूद नहीं होगी तो default value use होगी।

10. Object.keys(), values(), entries()

JavaScript में Object की सभी keys, values या key-value pairs को प्राप्त करने के लिए ये built-in methods उपयोग होते हैं।

(i) Object.keys()

यह method Object की सभी keys (property names) को Array के रूप में return करता है।

Syntax : Object.keys(objectName)

Example :

```
let student = {
  name : "Brajkant",
  age : 23,
  course : "PGDCA",
  city : "Kanpur"
};
let keys = Object.keys(student);
console.log(keys); // ['name', 'age', 'course', 'city']
```



(ii) Object.values()

यह method Object की सभी values को Array के रूप में return करता है।

Syntax : Object.values(objectName)

Example :

```
let values = Object.values(student);
console.log(values); // ['Brajkant', 23, 'PGDCA', 'Kanpur']
```

(iii) Object.entries()

यह method Object की सभी key-value pairs को nested Array के रूप में return करता है।

Syntax : Object.entries(objectName)

Example :

```
let entries = Object.entries(student);
console.log(entries);
// [['name', 'Brajkant'], ['age', 23],
//  ['course', 'PGDCA'], ['city', 'Kanpur']]
```

Summary Table :

Method	Return	Example Output
Object.keys()	Array of keys	['name', 'age', 'course', 'city']
Object.values()	Array of values	['Brajkant', 23, 'PGDCA', 'Kanpur']
Object.entries()	Array of key-value pairs	[['name', 'Brajkant'], ['age', 23], ...]

Key Points

- Object.keys() से सभी keys मिलती हैं।
- Object.values() से सभी values मिलती हैं।
- Object.entries() से key-value pairs मिलते हैं।
- ये methods Object data को easily iterate करने के लिए बहुत useful हैं।

11. for...in Loop with Object

for...in loop का उपयोग object की सभी properties पर iterate करने के लिए किया जाता है। यह हमें हर key को एक-एक करके access करने की सुविधा देता है।

Syntax :

```
for (let key in objectName) {  
    // code  
}
```

Example 1 : Basic Example

```
let student = {  
    name : "Braj Kant",  
    age : 23,  
    course : "PGDCA"  
};  
for (let key in student) {  
    console.log(key + " : " + student[key]);  
}
```

Output :

```
name : Braj Kant  
age : 23  
course : PGDCA
```

Example 2 : with if condition

```
let student = {  
    name : "Braj Kant",  
    age : 23,  
    course : "PGDCA",  
    city : "Kanpur"  
};  
for (let key in student) {  
    if (key !== "city") {  
        console.log(key + " : " + student[key]);  
    }  
}
```

Output :

```
name : Braj Kant  
age : 23  
course : PGDCA
```

Key Points

- for...in loop object की सभी enumerable properties को iterate करता है।
- यह object की keys को return करता है।
- Key से value प्राप्त करने के लिए object[key] का उपयोग किया जाता है।
- Array पर भी for...in काम करता है, लेकिन array के लिए for...of loop बेहतर होता है।

12. Spread Operator with Objects

Spread Operator (...) का उपयोग object की properties को copy करने, merge करने या new object बनाने के लिए किया जाता है। यह एक shallow copy बनाता है।

Syntax :

```
let newObj = { ...oldObj };
```

Example 1 : Copy Object

```
let student = {  
    name : "Braj Kant",  
    age : 23,  
    course : "PGDCA"  
};  
let copyStudent = { ...student };  
console.log(copyStudent);
```

Output :

```
{ name: 'Braj Kant', age: 23, course: 'PGDCA' }
```

Example 2 : Merge Two Objects

```
let personal = {  
    name : "Braj Kant",  
    age : 23  
};  
let academic = {  
    course : "PGDCA",  
    branch : "IT"  
};  
let student = { ...personal, ...academic };  
console.log(student);
```

Output :

```
{ name: 'Braj Kant', age: 23, course: 'PGDCA', branch: 'IT' }
```

Example 3 : Add or Override Properties

```
let student = {  
    name : "Braj Kant",  
    age : 23  
};  
let updatedStudent = { ...student, age: 24, city: "Kanpur" };  
console.log(updatedStudent);
```

Output :

```
{ name: 'Braj Kant', age: 24, city: 'Kanpur' }
```

Key Points

- Spread Operator (...) object की properties को expand करता है।
- यह shallow copy बनाता है (nested object को deep copy नहीं बनती)।
- Object को merge करने के लिए बहुत उपयोगी है।
- यदि same key होती है तो last value overwrite होती है।
- Spread Operator से हम आसानी से new object बना सकते हैं।

13. Object.assign()

Object.assign() method ka उपयोग ek ya adhik source object ki properties ko target object me copy (assign) karne ke liye होता है।

Syntax :

```
Object.assign(target, source1, source2, ...);
```

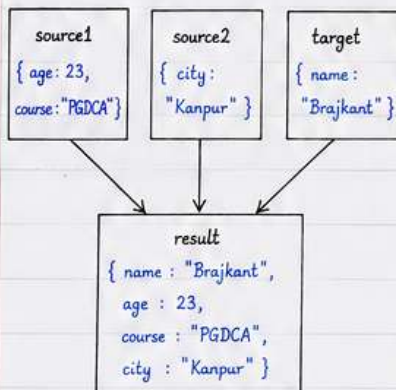
Example :

```
let target = { name: "Brajkant" };
let source1 = { age: 23, course: "PGDCA" };
let source2 = { city: "Kanpur" };

let result = Object.assign(target, source1, source2);
console.log(result);
```

Output :

```
{
  name: "Brajkant",
  age: 23,
  course: "PGDCA",
  city: "Kanpur"
}
```



Key Points

- Pehla argument target होता है जिसे properties copy hoti hain.
- Baad ke arguments source objects hote hain.
- Agar same property name milti hai to last source ki value overwrite kar deti hai.
- Ye method shallow copy karta hai (deep copy nahi karta).

14. Optional Chaining (?.)

Optional Chaining operator (?.) ka उपयोग undefined ya null values ke case me error aane se bachane ke liye होता है।

Isse nested object ki property ko safely access kiya जा सकता है।

Syntax :

```
object?.property
object?.property?.subProperty
```

Example 1 :

```
let student = {
  name: "Brajkant",
  address: {
    city: "Kanpur"
  }
};

console.log(student.address?.city);
// Kanpur

console.log(student.contact?.number);
// undefined (Error nahi aayega)
```

Example 2 :

```
let data = null;

console.log(data?.name); // undefined
console.log(data?.address?.city);
// undefined
```

Without Optional Chaining

```
let data = null;
console.log(data.name);
// Error: Cannot read properties of null
// (reading 'name')
```

Key Points

- Agar left side null ya undefined hai to expression evaluate nahi होता, aur undefined return होता है।
- Ye runtime errors ko prevent karta है।
- Deeply nested objects ke saath bahut useful है।

15. Constructor Functions

Constructor Function ek special type ka function होता है जिसका उपयोग multiple objects create karne के लिए किया जाता है। Iska naam usually PascalCase (first letter capital) में होता है।

Syntax :

```
function ConstructorName(param1, param2) {
  this.property1 = param1;
  this.property2 = param2;
  this.method = function() {
    // code
  }
}

let obj = new ConstructorName(value1, value2);
```

Example :

```
function Student(name, age, course) {
  this.name = name;
  this.age = age;
  this.course = course;
  this.displayInfo = function() {
    console.log("Name : " + this.name);
    console.log("Age : " + this.age);
    console.log("Course : " + this.course);
  }
}

let s1 = new Student("Brajkant", 23, "PGDCA");
let s2 = new Student("Aman", 22, "DCCN");
```

```
s1.displayInfo();
console.log("-----");
s2.displayInfo();
```

Output :

```
Name : Brajkant
Age : 23
Course : PGDCA
-----
Name : Aman
Age : 22
Course : DCCN
```

Key Points

- Constructor function का नाम PascalCase में होता है।
- new keyword से object create होता है।
- 'this' keyword constructor function के andar object को refer करता है।
- Alag-alag objects bana sakte hain with same structure.



TOPIC : HTML Events and calling JavaScript functions on Events.

Date : _____
Page No. : _____

1. HTML Events (Introduction)

- Event ek action hota hai jo user ya browser ke dwara trigger hota hai.
- HTML me hum events ka use karke web page ko interactive bana sakte hain.
- Jab koi event hota hai (jaise click, mouse move, key press etc.), to hum JavaScript function ko call kar sakte hain.
- Events ki wajah se web page user ke actions ko respond karta hai.

Syntax :

```
<element event="functionName()">  
</element>
```

- **element** → jis HTML element par event lagana hai.
- **event** → event ka naam (jaise onclick, onmouseover etc.)
- **functionName()** → wo JavaScript function jo execute hoga.

2. Common HTML Events

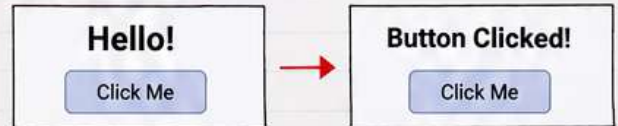
Event	Description	Example Use
onclick	Mouse se click karne par	Button click
ondblclick	Double click karne par	Open file
onmouseover	Mouse element ke upar aane par	Show tooltip
onmouseout	Mouse element se bahar jaane par	Hide tooltip
onchange	Input ki value change hone par	Form validation
oninput	Jab user kuch type kare tab	Live search
onkeydown	Key press karne par	Detect key
onkeyup	Key release karne par	Detect key release
onsubmit	Form submit karne par	Validate form
onload	Page load hone par	Initialize content
onresize	Window resize hone par	Responsive design
onfocus	Element par focus aane par	Highlight input
onblur	Element se focus hatne par	Remove highlight

3. onclick Event (Button Example)

Example :

```
<!DOCTYPE html>  
<html>  
<head><title>Click Event</title></head>  
<body>  
  <h2 id="msg">Hello!</h2>  
  <button onclick="showMessage()">Click Me</button>  
  <script>  
    function showMessage(){  
      document.getElementById("msg").innerText =  
        "Button Clicked!";  
    }  
  </script>  
</body>  
</html>
```

Output (Browser) :



4. onmouseover and onmouseout Event

Example :

```
<!DOCTYPE html>  
<html>  
<body>  
  <h3 id="text" onmouseover="changeColor()" onmouseout="resetColor()">Hover over me!</h3>  
  <script>  
    function changeColor(){  
      document.getElementById("text").style.color = "red";  
    }  
    function resetColor(){  
      document.getElementById("text").style.color = "black";  
    }  
  </script>  
</body>  
</html>
```

Output (Browser) :

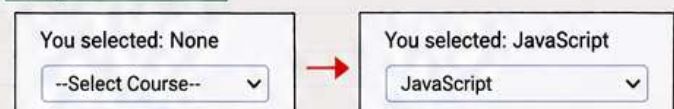


5. onchange Event (Select Box Example)

Example :

```
<!DOCTYPE html>  
<html>  
<body>  
  <p id="result">You selected: None</p>  
  <select onchange="showValue(this)">  
    <option value="--Select Course--">--Select Course--</option>  
    <option value="html">HTML</option>  
    <option value="css">CSS</option>  
    <option value="js">JavaScript</option>  
  </select>  
  <script>  
    function showValue(sel){  
      document.getElementById("result").innerText =  
        "You selected: " + sel.value;  
    }  
  </script>  
</body>  
</html>
```

Output (Browser) :



Key Points :

- Events se web page interactive banta hai.
- HTML me direct event attribute likh kar JavaScript function call kar sakte hain.
- Same function multiple elements par use kiya ja sakta hai.
- Events ka use form validation, animation, UI effects, user interaction etc. me hota hai.



BK LearnX
Learn | Practice | Achieve